

Logging in Clean Architecture

2.1 Logging and Clean Architecture Rule

Inner layers must NOT depend on outer layers

So:

- Domain → ❌ No logging
 - Application → ⚠️ Minimal, abstract logging
 - Infrastructure → ✅ Actual logging implementation
 - API → ✅ Request/response logging
-

2.2 Where Logging Belongs (Layer-wise)

Layer	Logging Responsibility
Domain	❌ No logging
Application	Business flow logging
Infrastructure	DB, external services
API	HTTP lifecycle

2.3 Logging in Application Layer (Use Case Level)

```
public class CreateProductHandler
{
    private readonly ILogger<CreateProductHandler> _logger;

    public CreateProductHandler(ILogger<CreateProductHandler> logger)
    {
        _logger = logger;
    }
}
```

```

public async Task Handle(CreateProductCommand command)
{
    _logger.LogInformation("Creating product: {Name}", command.Name);

    // business logic

    _logger.LogInformation("Product created successfully");
}
}

```

✓ This is acceptable because:

- Uses abstraction (ILogger)
- No dependency on Serilog or Seq

2.4 Logging in Infrastructure Layer

Example: EF Core repository

```

public class ProductRepository : IProductRepository
{
    private readonly ILogger<ProductRepository> _logger;

    public ProductRepository(ILogger<ProductRepository> logger)
    {
        _logger = logger;
    }

    public async Task AddAsync(Product product)

```

```
{  
    _logger.LogDebug("Saving product to database");  
    // EF Core Save  
}  
}
```

2.5 Logging in API Layer (Cross-Cutting)

Global Exception Logging Middleware

```
public class ExceptionMiddleware  
{  
    private readonly RequestDelegate _next;  
    private readonly ILogger<ExceptionMiddleware> _logger;  
  
    public ExceptionMiddleware(RequestDelegate next, ILogger<ExceptionMiddleware> logger)  
    {  
        _next = next;  
        _logger = logger;  
    }  
  
    public async Task Invoke(HttpContext context)  
    {  
        try  
        {  
            await _next(context);  
        }  
        catch (Exception ex)
```

```
{  
    _logger.LogError(ex, "Unhandled exception");  
    throw;  
}  
}  
}
```

2.6 Clean Architecture Logging Summary

- No logging in Domain
- Use ILogger<T> everywhere else
- Serilog configured only in API startup
- Infrastructure and Application remain framework-agnostic